

1. S - Single Responsibility Principle (SRP)

What is the Single Responsibility Principle (SRP)?

Code Example: Violating SRP

Problems with the Above Code

Code Example: Follows SRP

Key Benefits of the Refactored Code

Summary

▼ Resources

- Video → [1. SOLID Principles with Easy Examples \(Hindi\) | OOPs SOLID Principles - Low Level Design](#)

What is the Single Responsibility Principle (SRP)?

*Single Responsibility Principle(SRP) states that “**A class should have only ONE reason to change**”, meaning it should have ONE and ONLY ONE job or responsibility.*

Meaning, if the class has multiple jobs or responsibilities, changes to one responsibility might affect or break the other critical responsibilities, making the **code bloated, fragile, and harder to maintain**. Hence, the Single Responsibility Principle(SRP) focuses on restricting the concern to only a single responsibility.

Code Example: Violating SRP

Concept && Coding

```
1 public class Marker {
2     String name;
3     String color;
4     int price;
5     int year;
6
7     public Marker(String name, String color, int price, int year) {
8         this.name = name;
9         this.color = color;
10        this.price = price;
11        this.year = year;
12    }
13 }
14
15 // BAD: This class violates SRP by having multiple responsibilities
16 public class Invoice {
17     private Marker marker;
18     private int quantity;
19     private int total;
20
21     public Invoice(Marker marker, int quantity) {
22         this.marker = marker;
23         this.quantity = quantity;
24     }
25
26     // Responsibility 1: Calculate the total(business logic)
27     public void calculateTotal() {
28         System.out.println("Calculating total...");
29         this.total = this.marker.price * this.quantity;
30     }
31
32     // Responsibility 2: Database Operations
33     public void saveToDB() {
34         // Save the data into DB
```

```

35     System.out.println("Saving to DB...");
36 }
37
38 // Responsibility 3: Print the Invoice
39 public void printInvoice() {
40     // print the Invoice
41     System.out.println("Printing Invoice...");
42 }
43 }
44
45 // Usage example
46 public class Demo {
47     public static void main(String[] args) {
48         Invoice invoice = new Invoice(new Marker("name", "color", 10,
49         2020), 10);
50         invoice.calculateTotal();
51         invoice.saveToDB();
52         invoice.printInvoice();
53     }
54 }

```

Problems with the Above Code

The `Invoice` class has 3 different responsibilities

1. `calculateTotal()` → Responsibility 1: Calculate the total(business logic)
2. `saveToDB()` → Responsibility 2: Database Operations
3. `printInvoice()` → Responsibility 3: Print the Invoice

This violates SRP because:

- If the tax calculation rules change, we need to modify the `Invoice` class.
- If the database structure changes, we need to modify the `Invoice` class.
- If the printing requirement changes, we need to modify the `Invoice` class.

Code Example: Follows SRP

Here's the refactored code that follows the Single Responsibility Principle:

```

1
2 public class Marker {
3     String name;
4     String color;
5     int price;
6     int year;
7
8     public Marker(String name, String color, int price, int year) {
9         this.name = name;
10        this.color = color;
11        this.price = price;
12        this.year = year;
13    }
14 }
15
16 // GOOD: Following SRP - Each class has a single responsibility
17 // Responsibility: Managing Invoice data only
18 public class Invoice {
19
20     private Marker marker;
21     private int quantity;
22     private int total;
23
24     public Invoice(Marker marker, int quantity) {
25

```

```

26         this.marker = marker;
27         this.quantity = quantity;
28     }
29
30     // Responsibility 1: Calculate the total(business logic)
31     public void calculateTotal() {
32         System.out.println("Calculating total...");
33         this.total = this.marker.price * this.quantity;
34     }
35 }
36
37 // Responsibility 2: Managing Database Operations only
38 public class InvoiceDao {
39
40     Invoice invoice;
41
42     public InvoiceDao(Invoice invoice) {
43         this.invoice = invoice;
44     }
45
46     public void saveToDB() {
47         // Save into the DB the invoice
48         System.out.println("Saving to DB...");
49     }
50 }
51
52 // Responsibility 3: Printing the Invoice only
53 public class InvoicePrinter {
54
55     private Invoice invoice;
56
57     public InvoicePrinter(Invoice invoice) {
58         this.invoice = invoice;
59     }
60
61     public void print() {
62         // print the invoice
63         System.out.println("Printing Invoice...");
64     }
65 }
66
67 // Usage example showing how all classes work together
68 public class Demo {
69     public static void main(String[] args) {
70
71         // create the service objects
72         Invoice invoice = new Invoice(new Marker("name", "color", 10,
2020), 10);
73         InvoiceDao invoiceDao = new InvoiceDao(invoice);
74         InvoicePrinter invoicePrinter = new InvoicePrinter(invoice);
75
76         // Use the services
77         invoice.calculateTotal();
78         invoiceDao.saveToDB();
79         invoicePrinter.print();
80     }
81 }

```

Concept & Coding

Key Benefits of the Refactored Code

- Single Responsibility: Each class now has only one reason to change
 - **Invoice** : Only changes if the **Invoice** tax calculation rules change
 - **InvoiceDao** : Only changes if **InvoiceDao** database operations change
 - **InvoicePrinter** : Only changes if **InvoicePrinter** requirements change
- Better Maintainability: Changes to one functionality don't affect others.

- Improved Testability: Each class can be tested in isolation.
- Enhanced Reusability: Classes can be reused in different contexts. For example, different types of invoices(domestic and international) can use the same `InvoicePrinter` class.

Summary

The Single Responsibility Principle (SRP) is key to maintainable, testable, and flexible code. It ensures each class has one reason to change, fostering a modular system. SRP isn't about limiting classes to a single method, but rather about ensuring they have one responsibility, even with multiple methods that align with that purpose.

Concept && Coding

2. O - Open/Closed Principle (OCP)

What is the Open/Closed Principle (OCP)?

Code Example: Violating OCP

Problems with the Above Code

Code Example: Follows OCP

Key Benefits of the Refactored Code

Summary

▼ Resources

- Video → [1. SOLID Principles with Easy Examples \(Hindi\) | OOPs SOL ID Principles - Low Level Design](#)

What is the Open/Closed Principle (OCP)?

*Open/Closed Principle(OCP) states that “**A class should be open for extension but closed for modification**”.*

This means that new functionality can be added through extension (inheritance, composition, interfaces) rather than by modifying existing code. As existing code is already tested and deployed in production, adding modifications would introduce an additional risk and require additional testing effort.

Code Example: Violating OCP

Concept && Coding

```
1 // Responsibility: Managing Database Operations only
2 public class InvoiceDaoOld {
3
4     Invoice invoice;
5
6     public InvoiceDaoOld(Invoice invoice) {
7         this.invoice = invoice;
8     }
9
10    public void saveToDB() {
11        // Save into the DB the invoice
12        System.out.println("Saving to DB...");
13    }
14 }
```

```
1
2 // Responsibility: Managing Database Operations and File Operations
3 // BAD: This class violates OCP - needs modification for every new
4 // kind of save function
5 public class InvoiceDao {
6
7     Invoice invoice;
8
9     public InvoiceDao(Invoice invoice) {
10        this.invoice = invoice;
11    }
12
13    public void saveToDB() {
14        // Save into the DB the invoice
15        System.out.println("Saving to DB...");
16    }
17
18    // BAD: This design violates OCP
```

```

18 // Every time we add a new save function, we need to modify the
   existing InvoiceDao class
19 public void saveToFile() {
20     // Save into the file
21     System.out.println("Saving to file...");
22 }
23 }
24
25 // Usage example - showing the problem
26 public class Demo {
27     public static void main(String[] args) {
28         Invoice invoice = new Invoice(new Marker("name", "color", 10,
29         2020), 10);
30         invoice.calculateTotal();
31
32         InvoiceDao databaseFileSave = new InvoiceDao(invoice);
33         databaseFileSave.saveToDB(); // Save to DB
34         databaseFileSave.saveToFile(); // Save to File
35
36         // Problem: If we want to add a new function like
37         saveToMongoDB(),
38         // we need to modify InvoiceDao and all its derived classes(if
39         exists)
40         // This violates the "closed for modification" part of OCP
41     }
42 }

```

Problems with the Above Code

- **Modification Required:** Every time we add a new type of save functionality(let's say `saveToMongoDB()`), we must modify the existing `InvoiceDao` class.
- **Risk of Breaking Existing Code:** Changes might introduce bugs in working functionality, i.e., the `InvoiceDao` class that is already deployed in the production environment.
- **Testing Issues:** Need to retest all existing functionality when adding new save operations.

Code Example: Follows OCP

```

1 // GOOD: Following OCP using interfaces and polymorphism
2 public interface InvoiceDao {
3     public void save();
4 }
5
6 // Concrete implementation for DatabaseInvoiceDao
7 public class DatabaseInvoiceDao implements InvoiceDao {
8     Invoice invoice;
9
10    public DatabaseInvoiceDao(Invoice invoice) {
11        // set the invoice
12        this.invoice = invoice;
13    }
14
15    @Override
16    public void save() {
17        // Save into the DB the invoice
18        System.out.println("Saving to DB...");
19    }
20 }
21
22 // Concrete implementation for FileInvoiceDao
23 // NEW File Save Operation: An extension without modification!
24 public class FileInvoiceDao implements InvoiceDao {
25
26     Invoice invoice;
27
28     public FileInvoiceDao(Invoice invoice) {
29         // set the invoice

```

```

30     this.invoice = invoice;
31 }
32
33 @Override
34 public void save() {
35     // Save into the file the invoice
36     System.out.println("Saving to file...");
37 }
38 }
39
40 // Usage demonstrating OCP compliance
41 public class Demo {
42     public static void main(String[] args) {
43         Invoice invoice = new Invoice(new Marker("name", "color", 10,
44         2020), 10);
45         invoice.calculateTotal();
46
47         InvoiceDao databaseInvoiceDao = new
48         DatabaseInvoiceDao(invoice);
49         databaseInvoiceDao.save(); // Save to DB
50
51         InvoiceDao fileInvoiceDao = new FileInvoiceDao(invoice);
52         fileInvoiceDao.save(); // Save to File
53
54         // The system is:
55         // - OPEN for extension (new save functions can be added)
56         // - CLOSED for modification (existing code remains unchanged)
57     }
58 }

```

Key Benefits of the Refactored Code

- **Reduced risk:** Existing tested code remains unchanged.
- **Better maintainability:** New features don't break existing functionality.
- **Improved flexibility:** Easy to add new behaviors without touching existing code.
- **Enhanced testability:** New extensions can be tested independently.
- **Supports polymorphism:** Enables dynamic behavior through interfaces and inheritance.

Summary

By adhering to the Open/Closed Principle (OCP), developers can construct software architectures that are both adaptable and resilient, allowing them to evolve and expand without compromising existing functionalities.

This principle emphasizes the importance of abstraction and the creation of well-defined interfaces, which not only accommodate current requirements but also anticipate future enhancements.

Systems designed with OCP in mind enhance robustness and maintainability, increase the longevity and scalability of software solutions, and reduce the risk of introducing defects during changes.

3. L - Liskov Substitution Principle (LSP)

What is the Liskov Substitution Principle (LSP)?

Code Example 1: Violating LSP

Problems with the Above Code

Code Example 2: Follows LSP

Key Benefits of the Refactored Code

Code Example 2:

LSP Violation

LSP Solution

Summary

Resources

- Video → [1. SOLID Principles with Easy Examples \(Hindi\) | OOPs SOLID Principles - Low Level Design](#)
- Video 1.1 → [1.1 Liskov Substitution Principle \(LSP\) with Solution in Java - SOLID Principles of Low Level Design](#)

What is the Liskov Substitution Principle (LSP)?

Liskov Substitution Principle(LSP) states that the “**objects of a superclass should be replaceable with objects of its subclasses without breaking the application**”.

That means, if class B is a subtype of class A, then we should be able to replace objects of A with B without breaking the behaviour of the program. Subclass should extend the capability of the parent class, not narrow it down.

Code Example 1: Violating LSP

```
1
2 // BAD: This design violates LSP
3 public interface Bike {
4
5     void turnOnEngine();
6
7     void turnOffEngine();
8
9     void accelerate();
10
11    void applyBrakes();
12 }
13
14 // Subclass of Bike - implements all Bike class behavior
15 public class Motorcycle implements Bike {
16     String company;
17     boolean isEngineOn;
18     int speed;
19
20     public Motorcycle(String company, int speed) {
21         this.company = company;
22         this.speed = speed;
23     }
24
25     @Override
26     public void turnOnEngine() {
27         this.isEngineOn = true; // turn on the engine!
```



```

28         System.out.println("Engine is ON!");
29     }
30
31     @Override
32     public void turnOffEngine() {
33         this.isEngineOn = false; // turn off the engine!
34         System.out.println("Engine is OFF!");
35     }
36
37     @Override
38     public void accelerate() {
39         this.speed = this.speed + 10; // increase the speed
40         System.out.println("MotorCycle Speed: " + this.speed);
41     }
42
43     @Override
44     public void applyBrakes() {
45         this.speed = this.speed - 5; // decrease the speed
46         System.out.println("MotorCycle Speed: " + this.speed);
47     }
48 }
49
50 // This class violates LSP!
51 public class Bicycle implements Bike {
52     String brand;
53     Boolean hasGears;
54     int speed;
55
56     public Bicycle(String brand, Boolean hasGears, int speed) {
57         this.brand = brand;
58         this.hasGears = hasGears;
59         this.speed = speed;
60     }
61
62     // LSP Violation: Strengthening preconditions
63     // Bicycle changes the behavior of turnOnEngine
64     @Override
65     public void turnOnEngine() {
66         throw new AssertionError("Detail Message: Bicycle has no
engine!");
67     }
68     // Bicycle changes the behavior of turnOffEngine
69     @Override
70     public void turnOffEngine() {
71         throw new AssertionError("Detail Message: Bicycle has no
engine!");
72     }
73
74     @Override
75     public void accelerate() {
76         this.speed = this.speed + 10; // increase the speed
77         System.out.println("Bicycle Speed: " + this.speed);
78     }
79
80     @Override
81     public void applyBrakes() {
82         this.speed = this.speed - 5; // decrease the speed
83         System.out.println("Bicycle Speed: " + this.speed);
84     }
85 }
86
87
88 // Usage example - demonstrates the LSP violations
89 public class Demo {
90     public static void main(String[] args) {
91         // create the objects
92         Motorcycle motorCycle = new Motorcycle("HeroHonda", 10);
93         Bicycle bicycle = new Bicycle("Hercules", true, 10);
94
95         // use the objects

```

Concept & Coding

```

96      // Works fine with Motorcycle - implements all Bike class
    behavior
97      motorcycle.turnOnEngine();
98      motorcycle.accelerate();
99      motorcycle.applyBrakes();
100     motorcycle.turnOffEngine();
101     // Client expects to be able to see the same behavior with
    Bicycle
102     bicycle.turnOnEngine(); // fails to implement Bike class
    behavior
103     bicycle.accelerate();
104     bicycle.applyBrakes();
105     bicycle.turnOffEngine(); // fails to implement Bike class
    behavior
106     }
107 }

```

Problems with the Above Code

- Not all bikes have engines (eBikes and Bicycles), but the base class forces the `turnOnEngine()` method.
- Bicycle throws an exception, violating the expected contract.
- Client code cannot safely treat all Bike subclasses uniformly.

Code Example 2: Follows LSP

Here's the refactored code that follows the Liskov Substitution Principle:

```

1  abstract class Bike {
2
3      // All Bikes can do these things
4      public abstract void accelerate();
5
6      public abstract void applyBrakes();
7
8  }
9  public interface Engine {
10     void turnOnEngine();
11
12     void turnOffEngine();
13 }
14
15 // Subclass of Bike - implements all Bike class behavior
16 public class Motorcycle extends Bike implements Engine {
17     String company;
18     boolean isEngineOn;
19     int speed;
20
21     public Motorcycle(String company, int speed) {
22         this.company = company;
23         this.speed = speed;
24     }
25
26     @Override
27     public void turnOnEngine() {
28         this.isEngineOn = true; // turn on the engine!
29         System.out.println("Engine is ON!");
30     }
31
32     @Override
33     public void turnOffEngine() {
34         this.isEngineOn = false; // turn off the engine!
35         System.out.println("Engine is OFF!");
36     }
37
38     public void accelerate() {
39         this.speed = this.speed + 10; // increase the speed
40         System.out.println("MotorCycle Speed: " + this.speed);
41     }

```

```

42
43     public void applyBrakes() {
44         this.speed = this.speed - 5; // decrease the speed
45         System.out.println("MotorCycle Speed: " + this.speed);
46     }
47 }
48
49 // GOOD: Following LSP
50 // Subclass of Bike - implements all Bike class behavior
51 // As Bicycles do not have engines, we need not implement Engine
   interface
52 public class Bicycle extends Bike {
53
54     String brand;
55     Boolean hasGears;
56     int speed;
57
58     public Bicycle(String brand, Boolean hasGears, int speed) {
59         this.brand = brand;
60         this.hasGears = hasGears;
61         this.speed = speed;
62     }
63
64     @Override
65     public void accelerate() {
66         this.speed = this.speed + 10; // increase the speed
67         System.out.println("Bicycle Speed: " + this.speed);
68     }
69
70     @Override
71     public void applyBrakes() {
72         this.speed = this.speed - 5; // decrease the speed
73         System.out.println("Bicycle Speed: " + this.speed);
74     }
75 }
76
77 // Usage of LSP-compliant design
78 public class Demo {
79     public static void main(String[] args) {
80         // create the objects
81         Motorcycle motorcycle = new Motorcycle("HeroHonda", 10);
82         Bicycle bicycle = new Bicycle("Hercules", true, 10);
83
84         // use the objects
85         // Works fine with Motorcycle - implements all Bike class
   behavior
86         motorcycle.turnOnEngine();
87         motorcycle.accelerate();
88         motorcycle.applyBrakes();
89         motorcycle.turnOffEngine();
90         // Works fine with Bicycle - implements all Bike class
   behavior
91         bicycle.accelerate();
92         bicycle.applyBrakes();
93     }
94 }

```

Concept & Coding

Key Benefits of the Refactored Code

- **Prevents Fragile Code:** LSP helps avoid situations where subclass behavior violates the expectations set by the parent class.
- **Improves Flexibility:** By designing classes that can be substituted without breaking the parent contract.
- **Improved maintainability:** Changes to subclasses don't break existing code.
- **Promotes Code Reusability:** LSP facilitates the reuse of base classes and the creation of new subclasses without requiring modifications to existing code.

Code Example 2:

LSP Violation

```
1 public class Vehicle {
2     public Integer getNumberOfWheels() {
3         return 4;
4     }
5
6     public Boolean hasEngine() {
7         return true;
8     }
9 }
10
11 public class Motorcycle extends Vehicle {
12
13     public String getSpecifications() {
14         return "MotorCycle has " + this.getNumberOfWheels() + " wheels
and has engine: " + this.hasEngine();
15     }
16 }
17
18 public class Car extends Vehicle {
19     @Override
20     public Integer getNumberOfWheels() {
21         return 4;
22     }
23
24     public String getSpecifications() {
25         return "Car has " + this.getNumberOfWheels() + " wheels and
has engine: " + this.hasEngine();
26     }
27 }
28
29 public class Bicycle extends Vehicle {
30     public Boolean hasEngine() {
31         return null;
32     }
33 }
34
35 // Usage example - Violation of Liskov Substitution
36 public class ViolationDemo {
37     public static void main(String[] args) {
38         // Happy Flow
39         List<Vehicle> vehicleList = new ArrayList<>();
40         vehicleList.add(new Motorcycle());
41         vehicleList.add(new Car());
42         for (Vehicle vehicle : vehicleList) {
43             System.out.println(vehicle.hasEngine().toString());
44         }
45         // Add Bicycle - Violation of LSP
46         List<Vehicle> vehicleList2 = new ArrayList<>();
47         vehicleList2.add(new Motorcycle());
48         vehicleList2.add(new Car());
49         vehicleList2.add(new Bicycle());
50         for (Vehicle vehicle : vehicleList2) {
51             System.out.println(vehicle.hasEngine().toString()); //
throws NPE
52             // Client code will break for Bicycle
53         }
54     }
55 }
```

Concept && Coding

LSP Solution

```
1 public class Vehicle {
2     // Include generic methods
3     public Integer getNumberOfWheels() {
4         return 2;
5     }
6 }
```

```

6  }
7  public class Bicycle extends Vehicle {
8  }
9  public class EngineVehicle extends Vehicle {
10     public Boolean hasEngine() {
11         return true;
12     }
13 }
14 public class Motorcycle extends EngineVehicle {
15 }
16 }
17 public class Car extends EngineVehicle {
18     @Override
19     public Integer getNumberOfWheels() {
20         return 4;
21     }
22 }
23
24 public class SolutionDemo {
25     public static void main(String[] args) {
26         List<Vehicle> vehicleList = new ArrayList<>();
27         vehicleList.add(new Motorcycle());
28         vehicleList.add(new Car());
29         vehicleList.add(new Bicycle());
30         for (Vehicle vehicle: vehicleList) {
31             System.out.println(vehicle.getNumberOfWheels().toString());
32         }
33         List<EngineVehicle> vehicleList2 = new ArrayList<>();
34         vehicleList2.add(new Motorcycle());
35         vehicleList2.add(new Car());
36         // vehicleList2.add(new Bicycle()); // compile time error -
cannot add Bicycle
37         for (Vehicle vehicle: vehicleList2) {
38             // System.out.println(vehicle.hasEngine()); // compile
time error - cannot access hasEngine
39         }
40     }
41 }

```

Concept && Coding

Summary

LSP is vital because it upholds the integrity of your class hierarchy, ensuring that extending functionality or creating new subclasses won't disrupt the program's existing behavior. It keeps your code clean, modular, and safe to evolve.

4. I - Interface Segregation Principle (ISP)

What is the Interface Segregation Principle (ISP)?

Code Example: Violating ISP

Problems with the Above Code

Code Example: Follows ISP

Key Benefits of the Refactored Code

Summary

▼ Resources

- Video → [1. SOLID Principles with Easy Examples \(Hindi\) | OOPs SOLID Principles - Low Level Design](#)

What is the Interface Segregation Principle (ISP)?

Interface Segregation Principle (ISP) states that “clients should not be forced to depend on interfaces they don't use. Instead of having one large interface with many methods, it's better to have multiple smaller, focused interfaces”.

That means, Interfaces should be such that the client should NOT implement unnecessary functions they do not need.

Code Example: Violating ISP

Concept && Coding

```
1 // BAD: This class violates ISP
2 // This is a fat interface
3 // One large interface forcing all implementers to define unused
  methods
4 public interface RestaurantEmployee {
5
6     void prepareFood();
7
8     void decideMenu();
9
10    void serveFoodAndDrinks();
11
12    void takeOrder();
13
14    void cleanTheKitchen();
15 }
16
17
18 // BAD: This class violates ISP(clients shouldn't depend on unused
  interfaces)
19 // Bloated class with empty or error-throwing methods
20 // This Waiter is forced to implement methods it doesn't need
21 public class Waiter implements RestaurantEmployee {
22     @Override
23     public void takeOrder() {
24         System.out.println("Taking order...");
25     }
26
27     @Override
28     public void serveFoodAndDrinks() {
29         System.out.println("Serving food and drinks...");
30     }
31 }
```

```

32     @Override
33     public void cleanTheKitchen() {
34         // Forced to implement but doesn't make sense for a waiter
35         throw new AssertionError("Detail Message: Waiter cannot clean
the kitchen!");
36     }
37
38     @Override
39     public void prepareFood() {
40         // Forced to implement but doesn't make sense for a waiter
41         throw new AssertionError("Detail Message: Waiter cannot
prepare food!");
42     }
43
44     @Override
45     public void decideMenu() {
46         // Forced to implement but doesn't make sense for a waiter
47         throw new AssertionError("Detail Message: Waiter cannot decide
the menu!");
48     }
49
50 }
51
52
53 // Usage example - showing the problem
54 public class ViolationDemo {
55     public static void main(String[] args) {
56         Waiter waiter = new Waiter();
57         // Works fine
58         waiter.takeOrder();
59         waiter.serveFoodAndDrinks();
60
61         // These will throw exceptions
62         waiter.prepareFood(); // forced implementation
63         waiter.decideMenu(); // forced implementation
64         waiter.cleanTheKitchen(); // forced implementation
65     }
66 }

```

Concept & Coding

Problems with the Above Code

- Classes are forced to implement methods they don't support.
- Results in `AssertionError` being thrown.
- Code becomes bloated with empty or error-throwing methods.
- Violates the principle that clients shouldn't depend on unused interfaces.

Code Example: Follows ISP

```

1 // GOOD: This follows ISP - Multiple focused interfaces following ISP
2 public interface ChefTasks {
3     void prepareFood();
4
5     void decideMenu();
6 }
7
8 // GOOD: This follows ISP - Multiple focused interfaces following ISP
9 public interface WaiterTasks {
10     void serveFoodAndDrinks();
11
12     void takeOrder();
13 }
14
15 // GOOD: This follows ISP - Multiple focused interfaces following ISP
16 public interface MaintenanceTasks {
17     void cleanTheKitchen();
18
19     void reStockGroceries();

```

```

20 }
21
22 // GOOD: This class follows ISP
23 // Now classes only implement what they actually need - Clean
  implementations
24 public class Chef implements ChefTasks {
25
26     @Override
27     public void prepareFood() {
28         System.out.println("Preparing food...");
29     }
30
31     @Override
32     public void decideMenu() {
33         System.out.println("Deciding menu...");
34     }
35 }
36
37 // GOOD: This class follows ISP
38 // Now classes only implement what they actually need - Clean
  implementations
39 public class Waiter implements WaiterTasks {
40
41     @Override
42     public void serveFoodAndDrinks() {
43         System.out.println("Serving food and drinks...");
44     }
45
46     @Override
47     public void takeOrder() {
48         System.out.println("Taking order...");
49     }
50 }
51 // Usage example - Following ISP
52 public class SolutionDemo {
53     public static void main(String[] args) {
54         // Create the objects
55         // Now classes only implement what they actually support
56         Chef chef = new Chef();
57         Waiter waiter = new Waiter();
58
59         // Use the objects
60         // These work perfectly - no forced implementations
61         chef.prepareFood();
62         chef.decideMenu();
63         // These work perfectly - no forced implementations
64         waiter.takeOrder();
65         waiter.serveFoodAndDrinks();
66     }
67 }

```

Concept & Coding

Key Benefits of the Refactored Code

- Prevents bloated classes by having each class implement interfaces that it uses.
- No forced dependencies on irrelevant functionality.
- Cleaner, more maintainable design/code.
- Different Restaurant Employee entities can implement multiple interfaces only when they support those features.

Summary

The **Interface Segregation Principle (ISP)** helps maintain focused classes by ensuring they only implement the methods they need. To identify potential violations of the ISP, look for indicators such as low cohesion, large or “fat” interfaces, empty methods, and challenging testing scenarios. By recognizing these signs early, you can refactor your code to uphold a modular and flexible design.

Concept & Coding

5. D - Dependency Inversion Principle (DIP)

What is the Dependency Inversion Principle (DIP)?

Code Example: Violating DIP

Problems with the Above Code

Code Example: Follows DIP

Key Benefits of the Refactored Code

Summary

▼ Resources

- Video → [1. SOLID Principles with Easy Examples \(Hindi\) | OOPs SOLID Principles - Low Level Design](#)

What is the Dependency Inversion Principle (DIP)?

*The Dependency Inversion Principle (DIP) states that “**high-level components should not depend on low-level components directly; instead, they should depend on abstractions**”.*

In other words, classes should depend on interfaces rather than concrete classes. Additionally, abstractions should not depend on details; details should depend on abstractions.

Code Example: Violating DIP

Utility Classes

Concept && Coding

```
1 public interface Keyboard {
2     void getSpecifications();
3 }
4 public interface Mouse {
5     void getSpecifications();
6 }
7
8 // Low-level module - concrete implementation
9 public class WiredKeyboard implements Keyboard {
10     private final String connectionType;
11     private final String company;
12     private final String modelVersion;
13     private final String color;
14
15     public WiredKeyboard(String connectionType, String company,
16 String modelVersion, String color) {
17         this.connectionType = connectionType;
18         this.company = company;
19         this.modelVersion = modelVersion;
20         this.color = color;
21     }
22
23     public void getSpecifications() {
24         System.out.println("====> Wired Keyboard");
25         System.out.println("Connection Type: " + connectionType);
26         System.out.println("Company: " + company);
27         System.out.println("Model Version: " + modelVersion);
28         System.out.println("Color: " + color);
29     }
30 }
31
32 // Low-level module - concrete implementation
33 public class BluetoothKeyboard implements Keyboard {
```

```

33     private final String connectionType;
34     private final String company;
35     private final String modelVersion;
36     private final String color;
37
38     public BluetoothKeyboard(String connectionType, String company,
String modelVersion, String color) {
39         this.connectionType = connectionType;
40         this.company = company;
41         this.modelVersion = modelVersion;
42         this.color = color;
43     }
44
45     public void getSpecifications() {
46         System.out.println("==> Bluetooth Keyboard");
47         System.out.println("Connection Type: " + connectionType);
48         System.out.println("Company: " + company);
49         System.out.println("Model Version: " + modelVersion);
50         System.out.println("Color: " + color);
51     }
52 }
53
54 // Low-level module - concrete implementation
55 public class WiredMouse implements Mouse {
56     private final String connectionType;
57     private final String company;
58     private final String modelVersion;
59     private final String color;
60
61     public WiredMouse(String connectionType, String company, String
modelVersion, String color) {
62         this.connectionType = connectionType;
63         this.company = company;
64         this.modelVersion = modelVersion;
65         this.color = color;
66     }
67
68     public void getSpecifications() {
69         System.out.println("==> Wired Mouse");
70         System.out.println("Connection Type: " + connectionType);
71         System.out.println("Company: " + company);
72         System.out.println("Model Version: " + modelVersion);
73         System.out.println("Color: " + color);
74     }
75 }
76
77 // Low-level module - concrete implementation
78 public class BluetoothMouse implements Mouse {
79     private final String connectionType;
80     private final String company;
81     private final String modelVersion;
82     private final String color;
83
84     public BluetoothMouse(String connectionType, String company,
String modelVersion, String color) {
85         this.connectionType = connectionType;
86         this.company = company;
87         this.modelVersion = modelVersion;
88         this.color = color;
89     }
90
91     public void getSpecifications() {
92         System.out.println("==> Bluetooth Mouse");
93         System.out.println("Connection Type: " + connectionType);
94         System.out.println("Company: " + company);
95         System.out.println("Model Version: " + modelVersion);
96         System.out.println("Color: " + color);
97     }
98 }
99
100 // Low-level module - concrete implementation

```

Concept & Coding

```

101 public class BluetoothMouse implements Mouse {
102     private final String connectionType;
103     private final String company;
104     private final String modelVersion;
105     private final String color;
106
107     public BluetoothMouse(String connectionType, String company,
String modelVersion, String color) {
108         this.connectionType = connectionType;
109         this.company = company;
110         this.modelVersion = modelVersion;
111         this.color = color;
112     }
113
114     public void getSpecifications() {
115         System.out.println("==> Bluetooth Mouse");
116         System.out.println("Connection Type: " + connectionType);
117         System.out.println("Company: " + company);
118         System.out.println("Model Version: " + modelVersion);
119         System.out.println("Color: " + color);
120     }
121 }

```

```

1 // VIOLATION OF DIP
2 // High-level module directly depending on low-level module
3 public class MacBook {
4     private final WiredKeyboard keyboard;
5     private final WiredMouse mouse;
6
7     // Direct dependency on concrete class
8     public MacBook(WiredKeyboard wiredKeyboard, WiredMouse wiredMouse)
{
9         keyboard = wiredKeyboard; // Tight coupling
10        mouse = wiredMouse; // Tight coupling
11    }
12
13    public Mouse getMouse() {
14        return mouse;
15    }
16
17    public Keyboard getKeyboard() {
18        return keyboard;
19    }
20 }
21
22
23 public class DemoViolation {
24     public static void main(String[] args) {
25
26         // create keyboard and mouse objects
27         WiredKeyboard wiredKeyboard = new WiredKeyboard("USB", "Dell",
"F602", "Grey");
28         WiredMouse wiredMouse = new WiredMouse("USB", "Dell", "F602",
"Grey");
29         BluetoothKeyboard bluetoothKeyboard = new
BluetoothKeyboard("Bluetooth", "Logitech", "G102", "Black");
30         BluetoothMouse bluetoothMouse = new
BluetoothMouse("Bluetooth", "Logitech", "G102", "Black");
31
32         // create macbook
33         MacBook macBookWithWiredParts = new MacBook(wiredKeyboard,
wiredMouse);
34         macBookWithWiredParts.getKeyboard().getSpecifications();
35         macBookWithWiredParts.getMouse().getSpecifications();
36
37         // create macbook with bluetooth keyboard and mouse
38         // MacBook macBookWithBluetoothParts = new
MacBook(bluetoothKeyboard, bluetoothKeyboard);
39         // cannot create macbook with bluetooth keyboard and mouse
because

```

Concept && Coding

```

40      // macbook depends on wired keyboard and mouse - tight
    coupling - violation of DIP
41    }
42  }

```

Problems with the Above Code

- The **MacBook** is tightly coupled to the **WiredKeyboard** and the **WiredMouse**.
- Cannot create **MacBook** objects with different parts without modifying the **MacBook** class.
- Difficult to test **MacBook** in isolation
- High-level module **MacBook** depends on low-level module **WiredKeyboard** and the **WiredMouse**.

Code Example: Follows DIP

Utility Code remains the same

```

1  // Following DIP
2  // High-level module uses abstraction
3  public class MacBook {
4      private final Keyboard keyboard;
5      private final Mouse mouse;
6
7      // Abstraction - defines contract
8      // Dependency injection through constructor
9      public MacBook(Mouse mouse, Keyboard keyboard) {
10         this.keyboard = keyboard; // Works with any kind of keyboard
    and mouse
11         this.mouse = mouse;
12     }
13
14     public Mouse getMouse() {
15         return mouse;
16     }
17
18     public Keyboard getKeyboard() {
19         return keyboard;
20     }
21 }
22
23 // Usage example - following DIP
24 public class DemoSolution {
25     public static void main(String[] args) {
26         // create keyboard and mouse objects
27         WiredKeyboard wiredKeyboard = new WiredKeyboard("USB", "Dell",
    "F602", "Grey");
28         WiredMouse wiredMouse = new WiredMouse("USB", "Dell", "F602",
    "Grey");
29         BluetoothKeyboard bluetoothKeyboard = new
    BluetoothKeyboard("Bluetooth", "Logitech", "G102", "Black");
30         BluetoothMouse bluetoothMouse = new
    BluetoothMouse("Bluetooth", "Logitech", "G102", "Black");
31
32         // Can easily create macbooks with different combinations of
    keyboard and mouse
33
34         // create macbook with wired keyboard and mouse
35         MacBook macBookWithWiredParts = new MacBook(wiredMouse,
    wiredKeyboard);
36         macBookWithWiredParts.getKeyboard().getSpecifications();
37         macBookWithWiredParts.getMouse().getSpecifications();
38
39         // create macbook with bluetooth keyboard and mouse
40         MacBook macBookWithBluetoothParts = new
    MacBook(bluetoothMouse, bluetoothKeyboard);
41         macBookWithBluetoothParts.getKeyboard().getSpecifications();
42         macBookWithBluetoothParts.getMouse().getSpecifications();

```

Concept && Coding

```
43     }  
44 }
```

Key Benefits of the Refactored Code

- `BluetoothKeyboard` and `BluetoothMouse` depend only on the respective `Keyboard` and `Mouse` abstractions.
- Any different `MacBook` parts can be injected without changing the `MacBook`.
- Easy to mock for testing.
- New `MacBook` parts can be added without touching existing code.

Summary

The solution demonstrates true dependency inversion because the high-level module (`MacBook`) and low-level modules (`BluetoothKeyboard` , `BluetoothMouse` , `WiredKeyboard` , `WiredMouse`) both depend on the same abstraction (`Keyboard` and `Mouse`), and the abstraction doesn't depend on any concrete implementation details.

Concept && Coding